# Zephyr RTOS on QEMU

Build and run an embedded app on an emulated x86 board,
with no hardware and no admin rights

The finished app: a live dashboard that fills to 100% and prints "Done."

## What the app does

- Boots Zephyr v3.7 on the **qemu_x86** board inside QEMU
- Draws a three-line dashboard that updates in place once per second
- Fills a progress bar over 60 seconds, shows "Done.", then stops updating

## Tools used

- **Zephyr RTOS** 3.7: kernel, build system and drivers
- **west**: Zephyr's command-line tool for fetching, building and running
- **Zephyr SDK** 0.16.9: cross-compiler for x86
- **QEMU** 6.2: open-source emulator, unpacked into the home folder

## Project layout

```
~/zephyr-iot/
  CMakeLists.txt   build description
  prj.conf         Kconfig options
  VERSION          firmware version
  src/
    main.c         application code
```

# Setup

## 1. Install west in a Python environment

```
mkdir -p ~/zephyrproject
python3 -m venv ~/zephyrproject/.venv
source ~/zephyrproject/.venv/bin/activate
pip install west
```

## 2. Fetch Zephyr

```
cd ~/zephyrproject
west init
west update
west zephyr-export
pip install -r zephyr/scripts/requirements.txt
```

## 3. Get QEMU without root

Download the packages and unpack them into a folder you own.

```
mkdir -p ~/debs && cd ~/debs
apt-get download qemu-system-x86 qemu-system-data seabios
mkdir -p ~/qemu-local
dpkg-deb -x qemu-system-x86_*.deb  ~/qemu-local
dpkg-deb -x qemu-system-data_*.deb ~/qemu-local
dpkg-deb -x seabios_*.deb          ~/qemu-local
cp -n ~/qemu-local/usr/share/seabios/bios-256k.bin ~/qemu-local/usr/share/qemu/
export PATH=$HOME/qemu-local/usr/bin:$PATH
```

## 4. Point QEMU at its firmware folder

A small wrapper adds the firmware path, so west can start QEMU directly.

```
mv ~/qemu-local/usr/bin/qemu-system-i386 ~/qemu-local/usr/bin/qemu-system-i386.real
cat > ~/qemu-local/usr/bin/qemu-system-i386 << 'EOF'
#!/bin/sh
exec "$HOME/qemu-local/usr/bin/qemu-system-i386.real" \
     -L "$HOME/qemu-local/usr/share/qemu" "$@"
EOF
chmod +x ~/qemu-local/usr/bin/qemu-system-i386
```

# Project files

## CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20.0)
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
project(zephyr_iot)

target_sources(app PRIVATE src/main.c)
```

## prj.conf

```
CONFIG_LOG=y
CONFIG_HWINFO=y
```

## VERSION

Zephyr 3.7 reads the firmware version from this file.

```
VERSION_MAJOR = 1
VERSION_MINOR = 0
PATCHLEVEL = 0
VERSION_TWEAK = 0
```

## src/main.c

ANSI escape codes move the cursor back to fixed lines, so the screen updates in place instead of scrolling. `k_sleep()` paces the loop, and `K_FOREVER` parks the thread when the work is done.

```
#include <zephyr/kernel.h>
#include <zephyr/sys/printk.h>

#define BAR_WIDTH   40
#define CYCLE_SECS  60

int main(void)
{
    char bar[BAR_WIDTH + 1];
    uint32_t step = 0;

    printk("\033[2J");                                  /* clear screen   */
    printk("\033[1;1H=== Hello from Zephyr ===\033[K"); /* title, line 1  */

    while (1) {
        uint32_t filled = (step * BAR_WIDTH) / CYCLE_SECS;
        uint32_t pct    = (step * 100) / CYCLE_SECS;

        for (uint32_t i = 0; i < BAR_WIDTH; i++) {
            bar[i] = (i < filled) ? '#' : '-';
        }
        bar[BAR_WIDTH] = '\0';

        printk("\033[2;1HUptime   : %u s\033[K", step);          /* line 2 */
        printk("\033[3;1HProgress : [%s] %3u%%\033[K", bar, pct); /* line 3 */

        if (step >= CYCLE_SECS) {
            break;
        }
        k_sleep(K_SECONDS(1));
        step++;
    }

    printk("\033[4;1H\033[32mDone.\033[0m\033[K\n");   /* green, line 4 */

    while (1) {
        k_sleep(K_FOREVER);
    }
    return 0;
}
```

# Build and run

## Build for qemu_x86

```
source ~/zephyrproject/.venv/bin/activate
cd ~/zephyrproject/zephyr
west build -b qemu_x86 ~/zephyr-iot -d ~/zephyr-iot/build
```

## Run in QEMU

Exit QEMU with Ctrl+A, then X.

```
export PATH=$HOME/qemu-local/usr/bin:$PATH
west build -t run -d ~/zephyr-iot/build
```

## Result

# Next steps

- Persistent settings with a boot counter that survives reset
- Ethernet networking on qemu_x86 with host port forwarding
- MQTT telemetry to a broker on the host